Python Fundamentals - Part 3

Instructor Name

Agenda

- **Python Fundamentals - Week 3**

Topic	Time (min)
Review: Scope of Variables	20
Python OOP Concepts	60
Break	**10**
Custom Sort Functions in Python	30
Exception Handling	30
Break	**10**
File Handling	30
JSON Module in Python	30

Announcements

- Ask your questions through Zoom's "Q/A"
- Class activity will take place on Zoom's "Chat"
- Assignment Review on Thursday
- We will cover Python basics (See other classes for advanced topics)
- Resources (slides, codes, recording) will be shared after the class

About Me

- Siavash Alemzadeh
- Python and ML Instructor at Interview Kickstart
- PhD in Aero & Astro, University of Washington
- ML Engineer at Uber, Ex-Microsoft

Scope of Variables

Scope of Variables in Python

- **Scope:** The region of the code where a variable can be accessed and modified
- **Importance:** Proper scoping prevents unintended access or modification of variables

Local Variables

- **Local Variables:** Variables declared within a function or a code block
- Accessible only inside the function or block

Nonlocal Variables

- **NonLocal Variables:** Variables defined in an outer function but not in the global scope
- Used in nested functions to modify the outer function's variable

Global Variables

- **Global Variables:** Variables declared outside any function or block
- Accessible from any part of the code

Now, let’s see some code

Agenda

- **Python Fundamentals - Week 3**

Topic	Time (min)
Review: Scope of Variables	20
Python OOP Concepts	60
Break	**10**
Custom Sort Functions in Python	30
Exception Handling	30
Break	**10**
File Handling	30
JSON Module in Python	30

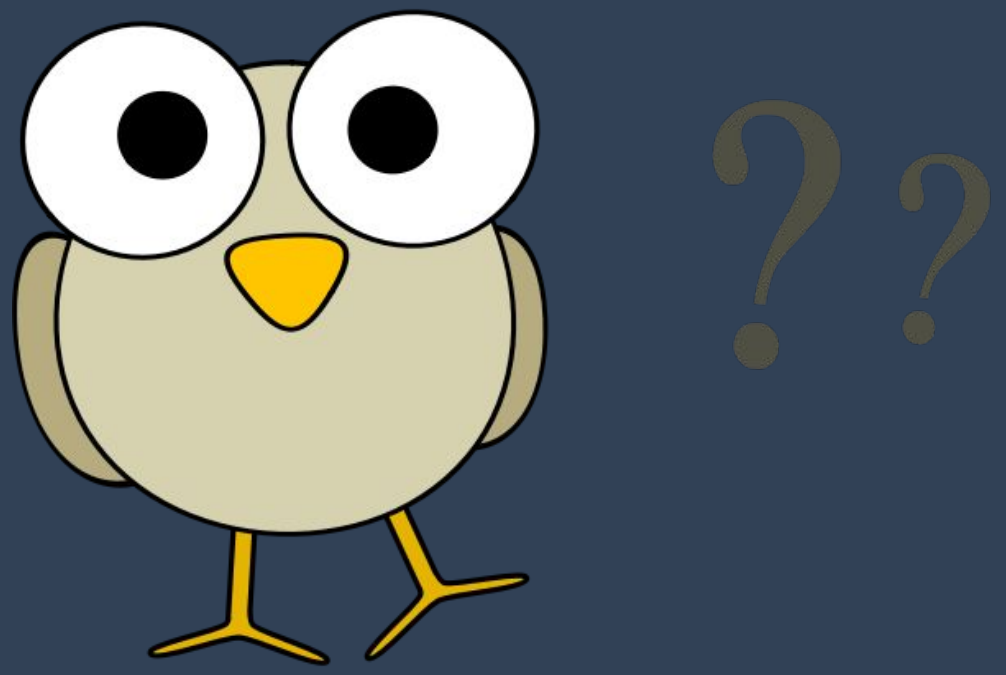
Object-Oriented Programming

Python OOP Concepts

- What is OOP?
  - A programming paradigm that uses objects to represent and manipulate data
  - Focus on creating reusable and modular code
  - Different style from procedural programming (Top-down design)

Python OOP Concepts

- Benefits of OOP in Python
  - Easier maintenance
  - Code reusability
  - Faster development
  - Inheritance
  - Encapsulation
  - Polymorphism



Now, let’s see some code

Inheritance

- **Definition:** A mechanism that allows one class to inherit properties and methods from another class
- Used for code reusability and reduced complexity

Now, let’s see some code

Encapsulation

- Bundling data (attributes) and methods (function) that operate on that data within a single unit (class)
- Blocks access to the internal state of an object and hence gives control over access to data
- Promotes modularity by bundling related data and functions together

Polymorphism

- **Definition:** The ability of different classes to share the same interface and be treated as objects of a common superclass

- **Benefits:** Flexibility in code, easier maintenance

Now, let’s see some code

Class Activity

Leetcode Problem

706. Design HashMap

Easy 4025 362 Add to List Share

Design a HashMap without using any built-in hash table libraries.

Implement the `MyHashMap` class:

- `MyHashMap()` initializes the object with an empty map.
- `void put(int key, int value)` inserts a `(key, value)` pair into the HashMap. If the `key` already exists in the map, update the corresponding `value`.
- `int get(int key)` returns the `value` to which the specified `key` is mapped, or `-1` if this map contains no mapping for the `key`.
- `void remove(key)` removes the `key` and its corresponding `value` if the map contains the mapping for the `key`.

Example 1:

```
Input
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
Output
[null, null, null, 1, -1, null, 1, null, -1]
```

Example 1:

```
Input
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
Output
[null, null, null, 1, -1, null, 1, null, -1]

Explanation
MyHashMap myHashMap = new MyHashMap();
myHashMap.put(1, 1); // The map is now [[1,1]]
myHashMap.put(2, 2); // The map is now [[1,1], [2,2]]
myHashMap.get(1);    // return 1, The map is now [[1,1], [2,2]]
myHashMap.get(3);    // return -1 (i.e., not found), The map is now [[1,1], [2,2]]
myHashMap.put(2, 1); // The map is now [[1,1], [2,1]] (i.e., update the existing
value)
myHashMap.get(2);    // return 1, The map is now [[1,1], [2,1]]
myHashMap.remove(2); // remove the mapping for 2, The map is now [[1,1]]
myHashMap.get(2);    // return -1 (i.e., not found), The map is now [[1,1]]
```

Solution

```python
class MyHashMap:

    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.myHashMap = dict()

    def put(self, key: int, value: int) -> None:
        """
        value will always be non-negative.
        """
        self.myHashMap[key] = value


    def get(self, key: int) -> int:
        """
        Returns the value to which the specified key is mapped, or -1 if this map contains no mapping
for the key
        """
        if key in self.myHashMap.keys():
            return self.myHashMap[key]
        return -1

    def remove(self, key: int) -> None:
        """
        Removes the mapping of the specified value key if this map contains a mapping for the key
        """
        self.myHashMap[key]=-1
```

Agenda

- **Python Fundamentals - Week 3**

Topic	Time (min)
Review: Scope of Variables ✓	20
Python OOP Concepts ✓	60
Break	**10**
Custom Sort Functions in Python	30
Exception Handling	30
Break	**10**
File Handling	30
JSON Module in Python	30

Starting in...
00:00

Custom Sort Functions

Custom Sort Functions in Python

- **sorted():** Returns a new sorted list from the specified iterable
- **sort():** Modifies the list in-place and returns None

Custom Sort Functions in Python

- **key:** Function to be called on each list element to determine the sorting order
- **reverse:** If set to True, the list is sorted in descending order

Now, let’s see some code

Agenda

- **Python Fundamentals - Week 3**

Topic	Time (min)
Review: Scope of Variables ✓	20
Python OOP Concepts ✓	60
Break	**10**
Custom Sort Functions in Python ✓	30
Exception Handling	30
Break	**10**
File Handling	30
JSON Module in Python	30

Exception Handling

Exception Handling

- **Definition:** Events that occur during the execution of a program when an error is encountered

- **Importance:** Proper handling prevents the program from crashing and allows for more graceful error management

Exception Handling

- Common Exceptions in Python
  - SyntaxError
  - TypeError
  - ValueError
  - FileNotFoundError

Python try...except Block

The `try...except` block is used to handle exceptions in Python. Here's the syntax of `try...except` block:

```
try:
    # code that may cause exception
except:
    # code to run when exception occurs
```

Here, we have placed the code that might generate an exception inside the `try` block. Every `try` block is followed by an `except` block.

When an exception occurs, it is caught by the `except` block. The `except` block cannot be used without the try block.

Exception Handling

```
Syntax:

try:
# Some Code...
except:
# optional block
# Handling of exception (if required)
else:
# execute if no exception
finally:
# Some code  .. (always executed)
```

Now, let’s see some code

Agenda

- **Python Fundamentals - Week 3**

Topic	Time (min)
Review: Scope of Variables ✓	20
Python OOP Concepts ✓	60
Break	**10**
Custom Sort Functions in Python ✓	30
Exception Handling ✓	30
Break	**10**
File Handling	30
JSON Module in Python	30

Starting in...
00:00

File Handling

File Handling

- **Definition:** The process of reading from or writing to a file
- **Importance:** reading/writing data from external sources, and generating reports

Different Modes to Open a File in Python

Mode	Description
`r`	Open a file for reading. (default)
`w`	Open a file for writing. Creates a new file if it does not exist or truncates the file if it exists.
`x`	Open a file for exclusive creation. If the file already exists, the operation fails.
`a`	Open a file for appending at the end of the file without truncating it. Creates a new file if it does not exist.

Opening Files

```
file = open("example. txt", "r")
content = file.read()
print (content)
file.close ()
```

Reading from Files

- **read():** Reads the entire content of the file
- **readline():** Reads a single line from the file
- **readlines():** Reads all lines and returns a list of lines

```
with open("example.txt","r") as file:
    for line in file.readlines():
        print(line.strip())
```

Writing to Files

- **write():** Writes a string to the file
- **writelines():** Writes a list of strings to the file

```
lines = ["First line\n", "Second line\n", "Third line\n"]
with open("output.txt","w") as file:
    file.writelines(lines)
```

Now, let’s see some code

Agenda

- **Python Fundamentals - Week 3**

Topic	Time (min)
Review: Scope of Variables ✓	20
Python OOP Concepts ✓	60
Break	**10**
Custom Sort Functions in Python ✓	30
Exception Handling ✓	30
Break	**10**
File Handling ✓	30
JSON Module in Python	30

JSON Module

JSON Module in Python

- **Definition:** JavaScript Object Notation (JSON) is a lightweight data-interchange format

- **Why JSON?**
  - Human-readable (text-based with key-value pair structure)
  - Lightweight (efficient for network communication)
  - Easy to parse and generate (Python dictionary)
  - Language-independent
  - Supports various data types

JSON Module Functions

- **json.dump():** Serialize Python object to a JSON-formatted file
- **json.dumps():** Serialize Python object to a JSON-formatted string
- **json.load():** Deserialize JSON data from a file into a Python object
- **json.loads():** Deserialize JSON data from a string into a Python object

Now, let’s see some code

Agenda

- **Python Fundamentals - Week 3**

Topic	Time (min)
Review: Scope of Variables ✓	20
Python OOP Concepts ✓	60
Break	**10**
Custom Sort Functions in Python ✓	30
Exception Handling ✓	30
Break	**10**
File Handling ✓	30
JSON Module in Python ✓	30

Questions?